iRAYPLE Acquisition Card

SDK Development Document (Python)



V1.2.0

Revision History

No.	Date	Version	Revision Content	Software Version
1	Jul. 2023	1.0.0	First Release.	2.4.0
2	Nov. 2023	1.1.0	Added the Interface of Notification Channel.	2.4.1
3	Mar. 2024	1.2.0	Added Matched Routines of Multi-camera and Image Saving Interface.	2.4.2

401674 Irayple 2025-01-18

Table of Contents

1 Overview.....5
1.1 Compatibility Introduction.....5
1.2 SDK Installation Directory.....5
1.2.1 SDK for Windows.....5
1.3 Related Resources of SDK (Python).....7
1.4 SDK Project Configuration.....7
2 Overall Calling Process of SDK.....9
3 Function Interface Calling of SDK.....10
3.1 System Operation Instruction.....10
3.1.1 Obtain Version Information.....10
3.1.2 Acquisition Card Enumeration (Find Device).....10
3.1.3 Camera Enumeration (Find Device).....11
3.2 Acquisition Card Operation.....11
3.2.1 Open and Close Acquisition Card.....11
3.2.2 Notification Channel Event Callback Registration.....12
3.3 Camera Operation Instruction.....13
3.3.1 Camera Open and Close.....13
3.3.2 Register Event Callback.....14
3.4 Streaming Data Operation.....14
3.4.1 Streaming Data Configuration.....14
3.4.2 Image Acquisition.....15
3.4.3 Acquire ImageData.....16
3.4.4 Operations on Streaming Data and Image Data.....18
3.5 Attribute Operation Instruction.....19
3.5.1 Configuration Download, Storage and Loading.....19
3.5.2 Attribute Determination, Acquisition and Setting.....20
3.5.2.1 Attribute Determination.....20
3.5.2.2 Attribute Acquisition.....22
3.5.2.3 Attribute Setting.....26
3.5.3 Command Attribute Execution.....28
3.5.4 Attribute Search and Usage.....28
3.5.4.1 Integer Attribute.....28
3.5.4.2 Floating Point Attribute.....30
3.5.4.3 Boolean Attribute.....32

4016[illegible]iRayple 2025-01-18

3.5.4.4 Enumeration Attribute 33
3.5.4.5 String Attribute 35
3.5.4.6 Command Attribute 36
3.6 Image Operation 36
3.6.1 Video Recording 36
3.6.2 Image Operation 37
4 Matched Routine Description 40
5 FAQ (Frequently Asked Question) 41
5.1 Only 8 Images are Obtained by Using the Python Interface 41

401674 Irayple 2025-01-10

1 Overview

1.1 Compatibility Introduction

Table 1-1 Operating System Requirement

OS Type	Version	Note
Windows	Win7 SP1 32-bit/64-bit	Ensure that OS version includes the patch of SP1.
	Win8 32-bit/64-bit	
	Win10 32-bit/64-bit	

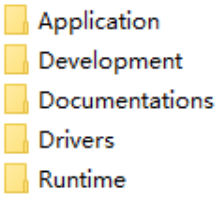
- Windows XP is no longer supported since SDK V2.2.5.
- The supported kernel version of Linux is 2.6.32 to 5.11.0 (Inclusive).
- Use the command of **uname –a** to check the kernel version number.

1.2 SDK Installation Directory

1.2.1 SDK for Windows

The installation directory structure of the SDK for Windows is as follows:

Figure 1-1 Installation directory structure



In V2.1.0, user can select the installation path.

Table 1-2 Installation directory structure description

Application	● **MViewer** is compatible with 32-bit and 64-bit. ● **CamTools** is compatible with 32-bit and 64-bit.
Development	**3rdPartyPlatformAdapter** It includes two plug-in dynamic libraries of third-party algorithm

	platform (Halcon and Sherlock).
	DOTENT DLL It includes **.Net** library file which is imported when user create the project.
	Include It includes the header file of the SDK interface, which explains the basic properties of various interfaces. The specific usage of header files is in the **samples folder**. The other is the extension definition of the header file of the SDK, which defines the SDK error code, the user-defined parameters, the data restriction of some parameters, the enumeration of various behaviors, and the data structure of various information.
	Lib It includes the lib files.
	Samples It includes the various types of demo file, such as **C#, Delphi, Lab View, MFC, Python, QT, VB** and **VC**.
Documentations	● **.Net** interface document ● **ActiveX** interface document ● **C** interface document ● **SDK** development document
Drivers	● The installation and uninstallation software of **DShowFilter**. ● The installation and uninstallation software of **GigE** high-performance driver. ● The installation and uninstallation software of **PCIeCL**. ● The installation and uninstallation software of **CLVSP** virtual serial port driver. ● The installation and uninstallation software of **USB** high-performance driver.
Runtime	The compilation dynamic link library when running the SDK. It includes the third-party **GenICam** library which is separated into two folders as 32-bit and 64-bit.

1.3 Related Resources of SDK (Python)

Table 1-3 SDK resource description

Category	Item	File Storage Folder
SDK Library	Header File	【SDK Installation Directory】\Development\Include\IMVFG\IMVFGApi.h \Development\Include\IMVFG\IMVFGDefines.h
	Runtime Library (32-bit)	【SDK Installation Directory】\Runtime \Win32\MVSDKmd.dll
	Runtime Library (64-bit)	【SDK Installation Directory】\Runtime \x64\MVSDKmd.dll
Python Routine	—	【SDK Installation Directory】\Development\Samples\Python\IMVFG

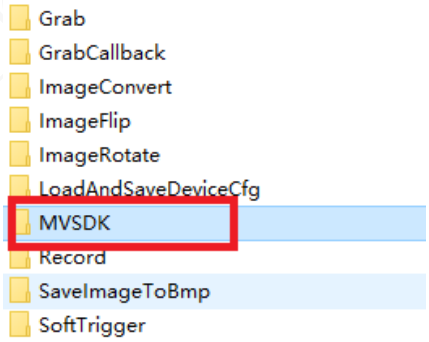
1.4 SDK Project Configuration

Take **x64** as an example to demonstrate the method of engineer configuration.

Copy Header File

【SDK Installation Directory】\Development\Sample\Python\IMVFG

Figure 1-2 Copy header file



Header File Import Module

Figure 1-3 Header file import module

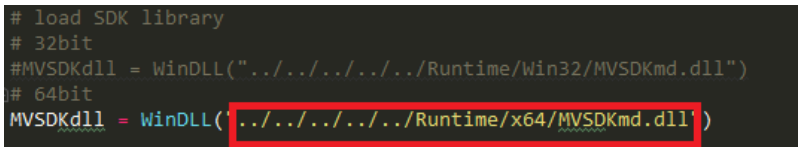
```
sys.path.append("../MVSDK")
from IMVFGApi import *
```

Import Dynamic Library

User can change the path loading the dynamic library in the header file of the IMVFGApi.py to the absolute path of MVSDKmd.dll.

【SDK Installation Directory】\Runtime\x64\MVSDKmd.dll

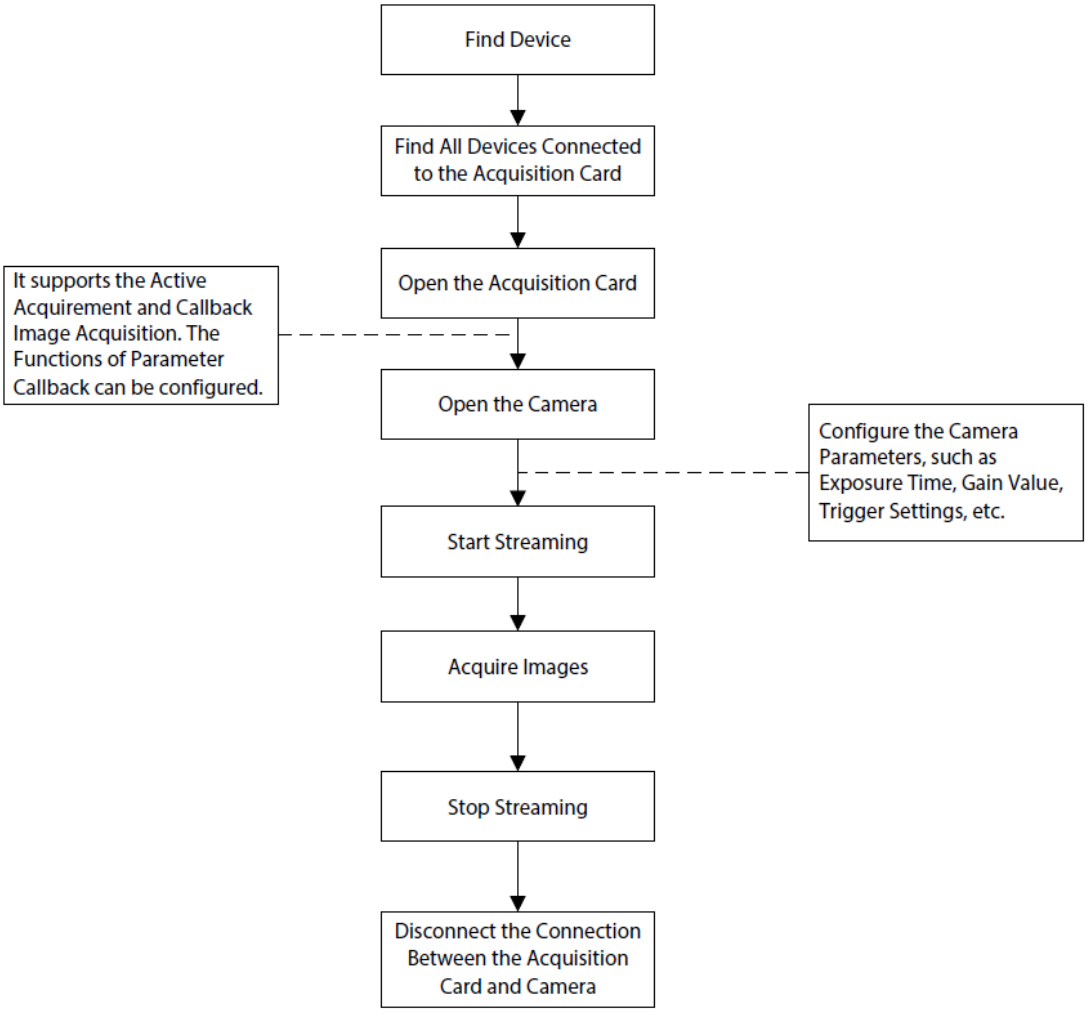
Figure 1-4 Dynamic library interface



401674 Irayple 2025-01-18

2 Overall Calling Process of SDK

Figure 2-1 SDK calling process



3 Function Interface Calling of SDK

- Directory of header files of SDK interface:
  \Development\Samples\Python\IMVFG\MVSDK\IMVFGApi.py
- Directory of the macro definition, enumeration, struct and callback function statement of SDK interface:
  \Development\Samples\Python\IMVFG\MVSDK\IMVFGDefines.py
- Object Description:
  - ◇ **card** refers to the routine object using the **Capture** type.
  - ◇ **cam** refers to the routine object using the **MvCamera**.

3.1 System Operation Instruction

3.1.1 Obtain Version Information

- Interface: IMV_FG_GetVersion()
  - ◇ Description: Get OS version. If succeeded, return version information. If failed, return **NULL**.
  - ◇ Reference Routine: All current routines can be referenced.

3.1.2 Acquisition Card Enumeration (Find Device)

- Interface: IMV_FG_EnumInterface(interfaceType, pInterfaceList)

Table 3-1 Parameter description

Parameter	I/O	Type	Description
pInterfaceList	OUT	IMV_FG_INTERFACE_INFO_LIST	Device list.
interfaceType	IN	int	The interface type to be enumerated can be combined arbitrarily, such as **interfaceTypeGige | interfaceTypeUsb3**.

  - ◇ Description: Acquisition card enumeration. If succeeded, return **IMV_FG_OK**. If failed, return error code.

- When **interfaceType = typeGevInterface,** enumerate all online devices connected through the **GigE** interface.
- When **interfaceType = typeU3vInterface,** enumerate all online devices connected through the **USB** interface.
- When **interfaceType = typeCLInterface,**enumerate the online devices connected through the **CameraLink** interface.
- When **interfaceType = typeCXPInterface,** enumerate all online devices connected through the **CXP** interface.
- The **interfaceType** of the **IMV_EnumDevices** support any combination of interface types. For example, if you need to enumerate all online devices connected through the **GigE** port and **USB 3.0** port, the **interfaceType** can be written as the **interfaceType = interfaceTypeGige | interfaceTypeUsb3**.

◇ Reference Routine: All current routines can be referenced.

3.1.3 Camera Enumeration (Find Device)

- Interface: IMV_FG_EnumDevices(nInterfaceType, pDeviceList)

Table 3-2 Parameter description

Parameter	I/O	Type	Description
nInterfaceType	IN	int	For the interface type to be enumerated, please refer to the acquisition card interface enumeration.
pDeviceList	OUT	IMV_FG_DEVICE_INFO_LIST	Device list.

◇ Description: Device Enumeration. Enumerate the cameras connected to the opened acquisition card.

◇ Reference Routine: All current routines can be referenced.

3.2 Acquisition Card Operation

3.2.1 Open and Close Acquisition Card

- Interface: IMV_FG_OpenInterface(self,nIndex)

Table 3-3 Parameter description

Parameter	I/O	Type	Description
self	IN	card	Acquisition card object.
nIndex	IN	int	Acquisition card serial number.

◇ Description: Acquisition card (connect the device according to the handle). If succeeded, return **IMV_FG_OK**. If failed, return error code.

◇ Reference Routine: All current routines can be referenced.

- Interface: IMV_FG_IsOpenInterface(self)

Table 3-4 Parameter description

Parameter	I/O	Type	Description
self	IN	card	Acquisition card object.

◇ Description: Determine whether the acquisition card is opened. If opened, return **True**. If close or offline, return **False**.

◇ Reference Routine: All current routines can be referenced.

- Interface: IMV_FG_CloseInterface(self)

Table 3-5 Parameter description

Parameter	I/O	Type	Description
self	IN	card	Acquisition card object.

◇ Description: Close device. (Close device according to the handle). If succeeded, return **IMV_FG_OK**. If failed, return error code.

◇ Reference Routine: All current routines can be referenced.

3.2.2 Notification Channel Event Callback Registration

- Interface: IMV_FG_SubscribeMsgChannelArg(self,proc,pUser)

Table 3-6 Parameter description

Parameter	I/O	Type	Description
self	IN	card	Acquisition card object.
proc	IN	IMV_FG_RotateImageParam	Device connection status event callback function.
pUser	IN	c_void_p	User-defined data, can be set to None.

◇ Description: Device connection status event callback registration. If succeeded, return **IMV_FG_OK**. If failed, return error code.



Only one callback function is supported, and after the device is turned off, the registration will be invalid. Please re-register the callback function after turning on the device.

◇ Reference Routine: None.

3.3 Camera Operation Instruction

3.3.1 Camera Open and Close

- Interface: IMV_FG_OpenDevice(self,mode,pIdentifier)

Table 3-7 Parameter description

Parameter	I/O	Type	Description
self	OUT	cam	Camera object.
mode	IN	IMV_FG_ECreateHandleMode	Handle creation mode.
pIdentifier	IN	bytes	Enable the camera identifier.

◇ Description: Open device. If succeeded, return **IMV_FG_OK**. If failed, return error code.

◇ Reference Routine: All current routines can be referenced.

- Interface: IMV_FG_OpenDeviceEx(self,mode,pIdentifier)

Table 3-8 Parameter description

Parameter	I/O	Type	Description
self	OUT	cam	Camera object.
mode	IN	IMV_FG_ECreateHandleMode	Handle creation mode.
pIdentifier	IN	bytes	Enable the camera identifier.

◇ Description: Open device. If succeeded, return **IMV_FG_OK**. If failed, return error code.

It supports the auto-synchronization of camera attributes to the acquisition card function, the attributes include width, height, tap and pixel format

◇ Reference Routine: All current routines can be referenced.

- Interface: IMV_FG_IsDeviceOpen(self)

Table 3-9 Parameter description

Parameter	I/O	Type	Description

self	IN	cam	Camera object.

◇ Description: Determine whether the device is opened. If opened, return **True**. If close or offline, return **False**.

◇ Reference Routine: None.

- Interface: IMV_FG_CloseDevice(self)

Table 3-10 Parameter description

Parameter	I/O	Type	Description
self	IN	cam	Camera object

◇ Description: Close device. (Close device according to the handle). If succeeded, return **IMV_FG_OK**. If failed, return error code.

◇ Reference Routine: All current routines can be referenced.

3.3.2 Register Event Callback

- Interface: IMV_FG_SubscribeConnectArg(self,proc,pUser)

Table 3-11 Parameter description

Parameter	I/O	Type	Description
self	IN	cam	Camera object.
proc	IN	IMV_FG_ConnectCallBack	Device connection status event callback function.
pUser	IN	c_void_p	User-defined data, can be set to None.

◇ Description: Camera connection status event callback registration. If succeeded, return **IMV_FG_OK**. If failed, return error code.

Only one callback function is supported, and after the device is turned off, the registration will be invalid. Please re-register the callback function after turning on the device.

◇ Reference Routine: None.

3.4 Streaming Data Operation

3.4.1 Streaming Data Configuration

- Interface: IMV_FG_SetBufferCount(self,nSize)

Table 3-12 Parameter description

Parameter	I/O	Type	Description
self	IN	card	Acquisition card object.
nSize	IN	int	Data buffer quantity.

◇ Description: Configure the buffer quantity of frame data. If succeeded, return **IMV_FG_OK**. If failed, return error code.



The configuration cannot be performed during the streaming.

◇ Reference Routine: None.

- Interface: IMV_FG_ClearFrameBuffer(self)

Table 3-13 Parameter description

Parameter	I/O	Type	Description
self	IN	card	Acquisition card object.

◇ Description: Clear the frame data buffer. If succeeded, return **IMV_FG_OK**. If failed, return error code.

◇ Reference Routine: None.

3.4.2 Image Acquisition

- Interface: IMV_FG_StartGrabbing(self)

Table 3-14 Parameter description

Parameter	I/O	Type	Description
self	IN	card	Acquisition card object.

◇ Description: Acquire images. If succeeded, return **IMV_FG_OK**. If failed, return error code.

◇ Reference Routine: **Grab**.

- Interface: IMV_FG_StartGrabbingEx(self,maxImagesGrabbed,strategy)

Table 3-15 Parameter description

Parameter	I/O	Type	Description
self	IN	card	Acquisition card object
maxImages Grabbed	IN	int	The max number of frames of acquired images. After the specified number of images is acquired, the image acquiring will stop. If the specified number is 0, it means that the image acquiring is continuously performed.

			 The parameter of **IMV_FG_StartGrabbing** is 0 by default.
strategy	IN	IMV_FG_EGrabStrategy	Image acquisition strategy (**IMV_FG_StartGrabbing** will use the **grabStrartegySequential** to acquire images).

◇ Description: Acquiring images. If succeeded, return **IMV_FG_OK**. If failed, return error code.

◇ Reference Routine: None.

- Interface: IMV_FG_IsGrabbing(self)

Table 3-16 Parameter description

Parameter	I/O	Type	Description
self	IN	card	Acquisition card object.

◇ Description: Determine whether the device is streaming. If streaming, return **True**. If not streaming, return **False**.

◇ Reference Routine: None.

- Interface: IMV_FG_StopGrabbing(self)

Table 3-17 Parameter description

Parameter	I/O	Type	Description
self	IN	card	Acquisition card object.

◇ Description: Stop Streaming. If succeeded, return **IMV_FG_OK**. If failed, return error code.

◇ Reference Routine: **Grab**.

3.4.3 Acquire ImageData

- Interface: IMV_FG_AttachGrabbing(self,proc,pUser)

Table 3-18 Parameter description

Parameter	I/O	Type	Description
self	IN	card	Acquisition card object.
proc	IN	IMV_FG_FrameCallBack	Frame data information callback function. Do not handle the time-consuming operations in this function,

			otherwise, it will affect the real-time performance of subsequent frame data.
pUser	IN	c_void_p	User-defined data, can be set to None.

◇ Description: Registered frame data callback function (asynchronous acquisition mechanism for frame data). If succeeded, return **IMV_FG_OK**. If failed, return error code.



- The asynchronous acquisition mechanism and synchronous acquisition mechanism (IMV_FG_GetFrame) are mutually exclusive, therefore, only one of them can be selected in the system for the same device.
- Only one callback function is supported, and after the device is turned off, the registration will be invalid. Please re-register the callback function after turning on the device.

◇ Reference Routine: **GrabCallback**, **Grab**.

- Interface: IMV_FG_GetFrame(self,pFrame,timeoutMS)

Table 3-19 Parameter description

Parameter	I/O	Type	Description
self	IN	card	Acquisition card object.
pFrame	OUT	IMV_FG_Frame	Frame Data Information.
timeoutMS	IN	int	The timeout value for obtaining a frame of image. The INFINITE mode means that it will continuously wait until a frame of image is received, otherwise, it will stop streaming (unit: ms).

◇ Description: Receive a frame of image (Synchronous acquisition mechanism for frame data). If succeeded, return **IMV_FG_OK**. If failed, return error code.



- This interface does not support the multi-thread callings.
- The synchronous and asynchronous acquisition mechanism (**IMV_FG_AttachGrabbing**) are mutually exclusive, therefore, only one of

them can be selected in the system for the same device.

- Acquiring images through releasing the internal cache. Use **IMV_FG_ReleaseFrame** to release the image buffer space.

◇ Reference Routine: **Grab**.

- Interface: IMV_FG_ReleaseFrame(self,pFrame)

Table 3-20 Parameter description

Parameter	I/O	Type	Description
self	IN	card	Acquisition card object.
pFrame	IN	IMV_FG_Frame	Frame data information.

◇ Description: Release the image buffer space. If succeeded, return **IMV_FG_OK**. If failed, return error code.

◇ Reference Routine: **Grab**.

3.4.4 Operations on Streaming Data and Image Data

- Interface: IMV_FG_CloneFrame(self,pFrame,pCloneFrame)

Table 3-21 Parameter description

Parameter	I/O	Type	Description
self	IN	card	Acquisition card object.
pFrame	IN	IMV_FG_Frame	Clone the source frame data information.
pCloneFrame	OUT	IMV_FG_Frame	New frame data information.

◇ Description: Frame data deep copy and clone. If succeeded, return **IMV_FG_OK**. If failed, return error code.

Use **IMV_FG_ReleaseFrame** to release the image buffer space.

◇ Reference Routine: None.

- Interface: IMV_FG_GetStatisticsInfo(self,pStreamStatsInfo)

Table 3-22 Parameter description

Parameter	I/O	Type	Description
self	IN	card	Acquisition card object.
pStreamStatsInfo	OUT	IMV_FG_StreamStatisticsInfo	Stream statistics information.

◇ Description: Obtain statistics information of streaming (after performing

IMV_FG_StartGrabbing / **IMV_FG_StartGrabbingEx** to call the obtain command). If succeeded, return **IMV_FG_OK**. If failed, return error code.

◇ Reference Routine: None.

- Interface: IMV_FG_ResetStatisticsInfo(self)

Table 3-23 Parameter description

Parameter	I/O	Type	Description
self	IN	card	Acquisition card object.

◇ Description: Reset statistics information of streaming (after performing **IMV_FG_StartGrabbing** / **IMV_FG_StartGrabbingEx** to call the resetting command). If succeeded, return **IMV_FG_OK**. If failed, return error code.

◇ Reference Routine: None.

3.5 Attribute Operation Instruction

3.5.1 Configuration Download, Storage and Loading

- Interface: IMV_FG_DownLoadGenICamXML(self,pFullFileName)

Table 3-24 Parameter description

Parameter	I/O	Type	Description
self	IN	card/cam	Acquisition Card/Camera object.
pFullFileName	IN	str	File saving path.

◇ Description: Download the device description XML file, and save it to the specified path, such as **D:\\xml.zip**. If succeeded, return **IMV_FG_OK**. If failed, return error code.

◇ Reference Routine: None.

- Interface: IMV_FG_SaveDeviceCfg(self,pFullFileName)

Table 3-25 Parameter description

Parameter	I/O	Type	Description
self	IN	card/cam	Acquisition card/camera object.
pFullFileName	IN	str	Full name (including saving path) of the exported device configuration file, such

			as **D:\\config.xml** or **D:\\config.mvcfg**.

◇ Description: Saving the device configuration file to the specified folder. The file with the same name that already exists will be overwritten. If succeeded, return **IMV_FG_OK**. If failed, return error code.

◇ Reference Routine: **LoadAndSaveDeviceCfg**

- Interface: IMV_FG_LoadDeviceCfg(self,pFullFileName,pErrorList)

Table 3-26 Parameter description

Parameter	I/O	Type	Description
self	IN	card/cam	Acquisition card/camera object.
pFullFileName	IN	str	Full name (including saving path) of the exported device configuration file, such as **D:\\config.xml** or **D:\\config.mvcfg**.
pErrorList	OUT	IMV_FG_ErrorList	The list of attribute name that failed to load. The upper limit of attributes that storage and loading failures is **IMV_MAX_ERROR_LIST_NUM**.

◇ Description: Load the xml configuration of device from files. If succeeded, return **IMV_FG_OK**. If failed, return error code.

◇ Reference Routine: **LoadAndSaveDeviceCfg**.

3.5.2 Attribute Determination, Acquisition and Setting

3.5.2.1 Attribute Determination

- Interface: IMV_FG_FeatureIsAvailable(self,pFeatureName)

Table 3-27 Parameter description

Parameter	I/O	Type	Description
self	IN	card/cam	Acquisition card/camera object.
pFeatureName	IN	str	Attribute name.

◇ Description: Determine whether the attribute is available. If available, return **True**. If unavailable, return **False**.

◇ Reference Routine: None.

- Interface: IMV_FG_FeatureIsReadable(self,pFeatureName)

Table 3-28 Parameter description

Parameter	I/O	Type	Description
self	IN	card/cam	Acquisition card/camera object.
pFeatureName	IN	str	Attribute name.

◇ Description: Determine whether the attribute is readable. If readable, return **True**. If unreadable, return **False**.

◇ Reference Routine: None.

- Interface: IMV_FG_FeatureIsWriteable(self,pFeatureName)

Table 3-29 Parameter description

Parameter	I/O	Type	Description
self	IN	card/cam	Acquisition card/camera object.
pFeatureName	IN	str	Attribute name.

◇ Description: Determine whether the attribute is writable. If writable, return **True**. If unwritable, return **False**.

◇ Reference Routine: None.

- Interface: IMV_FG_FeatureIsStreamable(self,pFeatureName)

Table 3-30 Parameter description

Parameter	I/O	Type	Description
self	IN	card/cam	Acquisition card/camera object.
pFeatureName	IN	str	Attribute name.

◇ Description: Determine whether the attribute is streamable. If streamable, return **True**. If unstreamable, return **False**.

◇ Reference Routine: None.

- Interface: IMV_FG_FeatureIsValid(self,pFeatureName)

Table 3-31 Parameter description

Parameter	I/O	Type	Description
self	IN	card/cam	Acquisition card/camera object.
pFeatureName	IN	str	Attribute name.

◇ Description: Determine whether the attribute is valid. If valid, return **True**. If invalid, return **False**.

◇ Reference Routine: None.

3.5.2.2 Attribute Acquisition

- Interface: IMV_FG_GetIntFeatureValue(self,pFeatureName,pIntValue)

Table 3-32 Parameter description

Parameter	I/O	Type	Description
self	IN	card/cam	Acquisition card/camera object.
pFeatureName	IN	str	Attribute name.
pIntValue	OUT	c_int	Integer attribute value.

◇ Description: Obtain the integer attribute value. If succeeded, return **IMV_FG_OK**. If failed, return error code.

◇ Reference Routine: None.

- Interface: IMV_FG_GetIntFeatureMin(self,pFeatureName,pIntValue)

Table 3-33 Parameter description

Parameter	I/O	Type	Description
self	IN	card/cam	Acquisition card/camera object.
pFeatureName	IN	str	Attribute name.
pIntValue	OUT	c_int	The min value that the integer can be configured.

◇ Description: Obtain the configurable min value of the integer attribute. If succeeded, return **IMV_FG_OK**. If failed, return error code.

◇ Reference Routine: None.

- Interface: IMV_FG_GetIntFeatureMax(self,pFeatureName,pIntValue)

Table 3-34 Parameter description

Parameter	I/O	Type	Description
self	IN	card/cam	Acquisition card/camera object.
pFeatureName	IN	str	Attribute name.
pIntValue	OUT	c_int	The configurable max value of the integer attribute.

◇ Description: Obtain the configurable max value of integer attribute. If succeeded, return **IMV_FG_OK**. If failed, return error code.

◇ Reference Routine: None.

- Interface: IMV_FG_GetIntFeatureInc(self,pFeatureName,pIntValue)

Table 3-35 Parameter description

Parameter	I/O	Type	Description
self	IN	card/cam	Acquisition card/camera object.
pFeatureName	IN	str	Attribute name.
pIntValue	OUT	c_int	Integer attribute step size.

◇ Description: Obtain the step size of the integer attribute. If succeeded, return **IMV_FG_OK**. If failed, return error code.

◇ Reference Routine: None.

- Interface: IMV_FG_GetDoubleFeatureValue(self,pFeatureName,pDoubleValue)

Table 3-36 Parameter description

Parameter	I/O	Type	Description
self	IN	card/cam	Acquisition card/camera object.
pFeatureName	IN	str	Attribute name.
pDoubleValue	OUT	c_doublr	Floating point attribute value.

◇ Description: Obtain the floating-point attribute value. If succeeded, return **IMV_FG_OK**. If failed, return error code.

◇ Reference Routine: None.

- Interface: IMV_FG_GetDoubleFeatureMin(self,pFeatureName,pDoubleValue)

Table 3-37 Parameter description

Parameter	I/O	Type	Description
self	IN	card/cam	Acquisition card/camera object.
pFeatureName	IN	str	Attribute name.
pDoubleValue	OUT	c_double	The configurable min value of the floating-point attribute.

◇ Description: Obtain the configurable min value of the floating-point attribute. If succeeded, return **IMV_FG_OK**. If failed, return error code.

◇ Reference Routine: None.

- Interface: IMV_FG_GetDoubleFeatureMax(self,pFeatureName,pDoubleValue)

Table 3-38 Parameter description

Parameter	I/O	Type	Description

self	IN	card/cam	Acquisition card/camera object.
pFeatureName	IN	str	Attribute name.
pDoubleValue	OUT	c_double	The configurable max value of the floating-point attribute.

◇ Description: Obtain the configurable max value of floating-point attribute. If succeeded, return **IMV_FG_OK**. If failed, return error code.

◇ Reference Routine: None.

- Interface: IMV_FG_GetBoolFeatureValue(self,pFeatureName,pBoolValue)

Table 3-39 Parameter description

Parameter	I/O	Type	Description
self	IN	card/cam	Acquisition card/camera object.
pFeatureName	IN	str	Attribute name.
pBoolValue	OUT	c_bool	Boolean attribute value.

◇ Description: Obtain the Boolean attribute value. If succeeded, return **IMV_FG_OK**. If failed, return error code.

◇ Reference Routine: None.

- Interface: IMV_FG_GetStringFeatureValue(self,pFeatureName,pStringValue)

Table 3-40 Parameter description

Parameter	I/O	Type	Description
self	IN	card/cam	Acquisition card/camera object.
pFeatureName	IN	str	Attribute name.
pStringValue	OUT	IMV_FG_String	String attribute value.

◇ Description: Obtain the string attribute value. If succeeded, return **IMV_FG_OK**. If failed, return error code.

◇ Reference Routine: None.

- Interface: IMV_FG_GetEnumFeatureValue(self,pFeatureName,pEnumValue)

Table 3-41 Parameter description

Parameter	I/O	Type	Description
self	IN	card/cam	Acquisition card/camera object.
pFeatureName	IN	str	Attribute name.

pEnumValue	OUT	c_int	Enumeration attribute value.

◇ Description: Obtain the enumeration attribute value. If succeeded, return **IMV_FG_OK**. If failed, return error code.

◇ Reference Routine: None.

- Interface: IMV_FG_GetEnumFeatureSymbol(self,pFeatureName,pEnumSymbol)

Table 3-42 Parameter description

Parameter	I/O	Type	Description
self	IN	card/cam	Acquisition card/camera object.
pFeatureName	IN	str	Attribute name.
pEnumSymbol	OUT	IMV_FG_String	Enumeration symbol value.

◇ Description: Obtain the enumeration symbol value. If succeeded, return **IMV_FG_OK**. If failed, return error code.

◇ Reference Routine: None.

- Interface: IMV_FG_GetEnumFeatureEntryNum(self,pFeatureName,pEntryNum)

Table 3-43 Parameter description

Parameter	I/O	Type	Description
self	IN	card/cam	Acquisition card/camera object.
pFeatureName	IN	str	Attribute name.
pEntryNum	OUT	c_int	The configurable number of the enumeration attribute value.

◇ Description: Obtain the configurable number of the enumeration attribute value. If succeeded, return **IMV_FG_OK**. If failed, return error code.

◇ Reference Routine: None.

- Interface: IMV_FG_GetEnumFeatureEntrys(self,pFeatureName,pEnumEntryList)

Table 3-44 Parameter description

Parameter	I/O	Type	Description
self	IN	card/cam	Acquisition card/camera object.
pFeatureName	IN	str	Attribute name.
pEnumEntryList	OUT	IMV_FG_EnumEntryList	The enumeration value list

			can be configured in the enumeration attribute value.

◇ Description: Obtain the enumeration value list of the enumeration attribute value. If succeeded, return **IMV_FG_OK**. If failed, return error code.

◇ Reference Routine: None.

3.5.2.3 Attribute Setting

- Interface: IMV_FG_SetIntFeatureValue(self,pFeatureName,pIntValue)

Table 3-45 Parameter description

Parameter	I/O	Type	Description
self	IN	card/cam	Acquisition card/camera object.
pFeatureName	IN	str	Attribute name.
pIntValue	IN	int	The integer attribute value to be configured.

◇ Description: Obtain the integer attribute value. If succeeded, return **IMV_FG_OK**. If failed, return error code.

◇ Reference Routine: None.

- Interface: IMV_FG_SetDoubleFeatureValue(self,pFeatureName,doubleValue)

Table 3-46 Parameter description

Parameter	I/O	Type	Description
self	IN	card/cam	Acquisition card/camera object.
pFeatureName	IN	str	Attribute name.
doubleValue	IN	double	Floating point attribute value which is to be configured.

◇ Description: Configure the floating-point attribute value. If succeeded, return **IMV_FG_OK**. If failed, return error code.

◇ Reference Routine: None.

- Interface: IMV_FG_SetBoolFeatureValue(self,pFeatureName,boolValue)

Table 3-47 Parameter description

Parameter	I/O	Type	Description

self	IN	card/cam	Acquisition card/camera object.
pFeatureName	IN	str	Attribute name.
boolValue	IN	bool	Boolean attribute value which is to be configured.

◇ Description: Configure the Boolean attribute value. If succeeded, return **IMV_FG_OK**. If failed, return error code.

◇ Reference Routine: None.

- Interface: IMV_FG_SetEnumFeatureValue(self,pFeatureName,enumValue)

Table 3-48 Parameter description

Parameter	I/O	Type	Description
self	IN	card/cam	Acquisition card/camera object.
pFeatureName	IN	str	Attribute name.
enumValue	IN	int	Enumeration attribute value which is to be configured.

◇ Description: Configure the enumeration attribute value. If succeeded, return **IMV_FG_OK**. If failed, return error code.

◇ Reference Routine: None.

- Interface: IMV_FG_SetEnumFeatureSymbol(self,pFeatureName,pEnumSymbol)

Table 3-49 Parameter description

Parameter	I/O	Type	Description
self	IN	card/cam	Acquisition card/camera object.
pFeatureName	IN	str	Attribute name.
pEnumSymbol	IN	str	Enumeration symbol value which is to be configured.

◇ Description: Configure the enumeration symbol value. If succeeded, return **IMV_FG_OK**. If failed, return error code.

◇ Reference Routine: **SoftTrigger**.

- Interface: IMV_FG_SetStringFeatureValue(self,pFeatureName,pStringValue)

Table 3-50 Parameter description

Parameter	I/O	Type	Description
self	IN	card/cam	Acquisition card/camera object.

pFeatureName	IN	str	Attribute name.
pStringValue	IN	str	String attribute value which is to be configured.

◇ Description: Configure the string attribute value. If succeeded, return **IMV_FG_OK**. If failed, return error code.

◇ Reference Routine: None.

3.5.3 Command Attribute Execution

- Interface: IMV_FG_ExecuteCommandFeature(self,pFeatureName)

Table 3-51 Parameter description

Parameter	I/O	Type	Description
self	IN	card/cam	Acquisition card/camera object.
pFeatureName	IN	str	Attribute name.

◇ Description: Execute the command attribute. If succeeded, return **IMV_FG_OK**. If failed, return error code.

◇ Reference Routine: **SoftTrigger**.

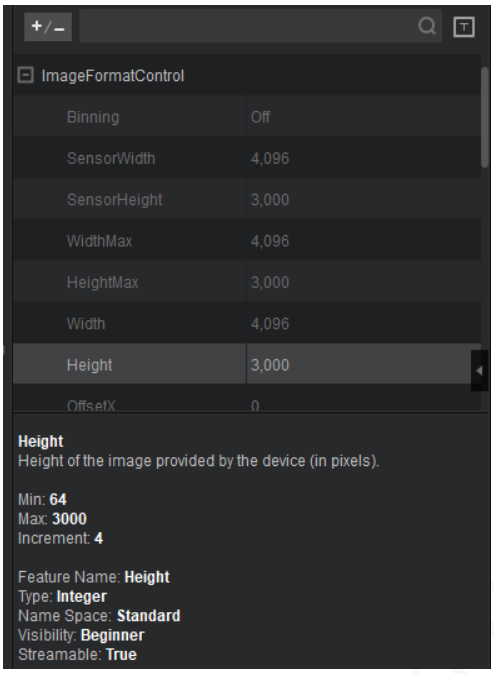
3.5.4 Attribute Search and Usage

User can search the attribute on the client, and learn the detailed information of the attribute, such as data type, value range, etc.

3.5.4.1 Integer Attribute

If user needs to obtain the pixel height information of the camera, find and click the Height Attribute. The attribute information of it will be displayed in the below. The data type of the Height Attribute is integer as shown in the figure.

Figure 3-1 Integer attribute of Height



User can obtain and modify the found attributes.

```
heightValue = c_int64_t(0)
heightMinValue = c_int64_t(0)
heightMaxValue = c_int64_t(0)
incrementValue = c_int64_t(0)

// Obtain attribute value.
ret = cam.IMV_FG_GetIntFeatureValue("Height", heightValue)
if (IMV_FG_OK != ret):
    print("Get feature value failed! ErrorCode[%d]"% ret)
    return ret

// Obtain the configurable min value of attribute. If there is no min value, return the error code.
ret = cam.IMV_FG_GetIntFeatureMin("Height", heightMinValue)
if (IMV_FG_OK != ret):
    print("Get feature minimum value failed! ErrorCode[%d]"% ret)
    return ret

// Obtain the configurable max value of attribute.If there is no max value, return the error code.
ret = cam.IMV_FG_GetIntFeatureMax("Height", heightMaxValue)
```

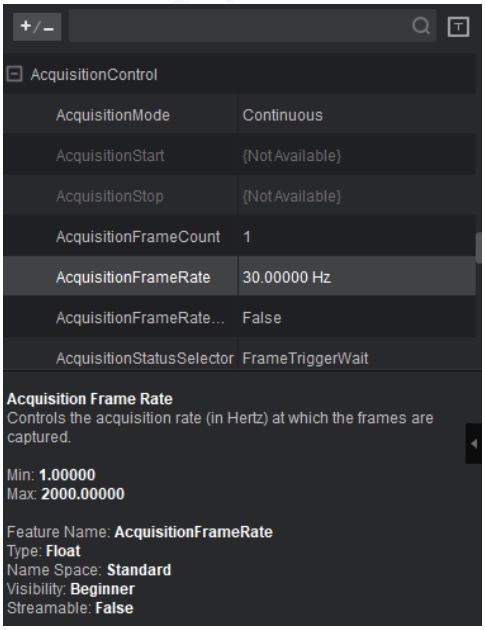
```
if (IMV_FG_OK != ret):
    print("Get feature maximum value failed! ErrorCode[%d]"% ret)
    return ret

// Obtain the attribute step size. If there is no step-size value, return the error code.
ret = cam.IMV_FG_GetIntFeatureInc("Height", incrementValue)
if (IMV_FG_OK != ret):
    print("Get feature increment value failed! ErrorCode[%d]"% ret)
    return ret

// Configure the attribute value.
ret = cam.IMV_FG_SetIntFeatureValue("Height", heightValue.value)
if (IMV_FG_OK != ret):
    print("Set feature value failed! ErrorCode[%d]"% ret)
    return ret
```

3.5.4.2 Floating Point Attribute

Figure 3-2 Floating point attribute




```
framerate = c_double(0)
frameratemin = c_double(0)
frameratemax = c_double(0)

// Obtain attribute value.
```

```
    ret = cam.IMV_FG_GetDoubleFeatureValue ("AcquisitionFrameRate", framerate)
    if (IMV_FG_OK != ret):
        print("Get feature value failed! ErrorCode[%d]"% ret)
        return ret

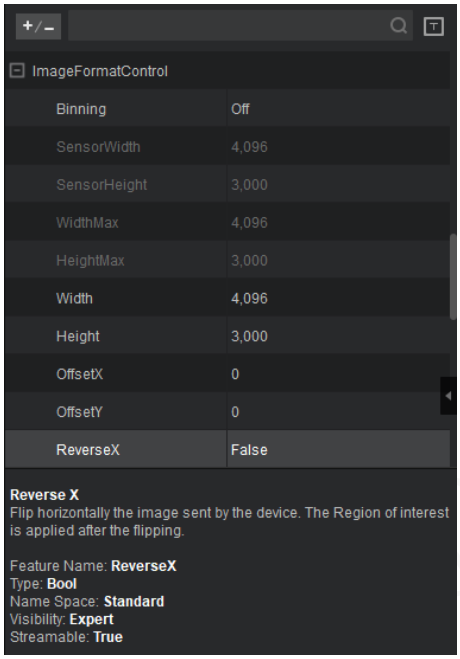
    // Obtain the configurable min value of attribute. If there is no min value, return the
error code.
    ret = cam.IMV_FG_GetDoubleFeatureMin("AcquisitionFrameRate", frameratemin)
    if (IMV_FG_OK != ret):
        print("Get feature minimum value failed! ErrorCode[%d]"% ret)
        return ret

    // Obtain the configurable max value of attribute. If there is no max value, return the
error code.
    ret = cam.IMV_FG_GetDoubleFeatureMax("AcquisitionFrameRate", frameratemax)
    if (IMV_FG_OK != ret):
        print("Get feature maximum value failed! ErrorCode[%d]"% ret)
        return ret
    // Configure the attribute value.
    ret = cam.IMV_FG_SetDoubleFeatureValue("AcquisitionFrameRate", framerate.value)
    if (IMV_FG_OK != ret):
        print("Set feature value failed! ErrorCode[%d]"% ret)
        return ret
```

401674 iRayple 2025-01-18

3.5.4.3 Boolean Attribute

Figure 3-3 Boolean attribute



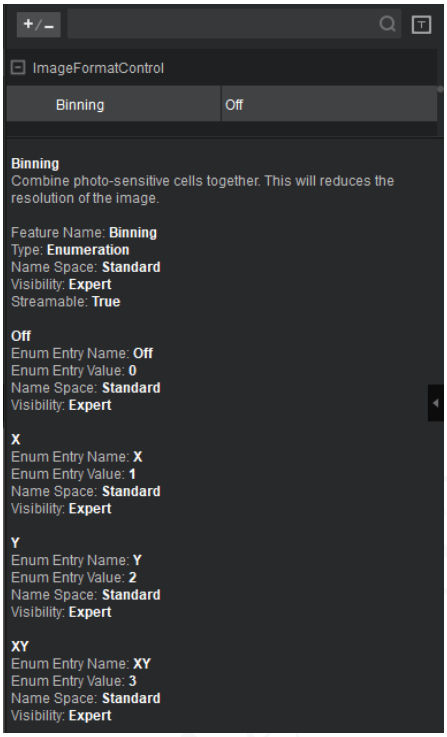

```
reversevalue = c_bool(0)

// Obtain attribute value.
ret = cam.IMV_FG_GetBoolFeatureValue ("ReverseX", reversevalue)
if (IMV_FG_OK != ret):
        print("Get feature value failed! ErrorCode[%d]"% ret)
        return ret

// Configure the attribute value.
ret = cam.IMV_FG_SetBoolFeatureValue("ReverseX", reversevalue.value)
if (IMV_FG_OK != ret):
        print("Set feature value failed! ErrorCode[%d]"% ret)
        return ret
```

3.5.4.4 Enumeration Attribute

Figure 3-4 Enumeration attribute




```
enumValue = c_uint64(0)
  pEnumSymbol = IMV_FG_String()
  pEntryNum = c_uint(0)
  pEnumEntryList = IMV_FG_EnumEntryList()

// Obtain the configurable enumeration value list of attribute.
ret = cam.IMV_FG_GetEnumFeatureValue("Binning", pEnumEntryList)
if (IMV_FG_OK != ret):
      print("Get feature value failed! ErrorCode[%d]"% ret)
      return ret

//Obtain the number of configurable enumeration values of attribute
ret = cam.IMV_FG_GetEnumFeatureEntryNum("Binning", pEntryNum)
if (IMV_FG_OK != ret):
      print("Get feature value failed! ErrorCode[%d]"% ret)
      return ret

// Obtain the attribute symbol value.
```

```
    ret = cam.IMV_FG_GetEnumFeatureSymbol("Binning", pEnumSymbol)
    if (IMV_FG_OK != ret):
        print("Get feature value failed! ErrorCode[%d]"% ret)
        return ret

    // Obtain attribute value.
    ret = cam.IMV_FG_GetIntFeatureInc("Binning", enumValue)
    if (IMV_FG_OK != ret):
        print("Get feature value failed! ErrorCode[%d]"% ret)
        return ret

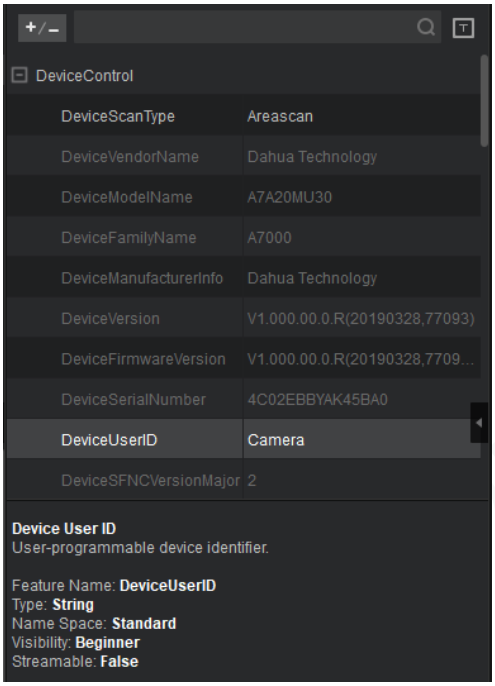
    // Configure attribute symbol value.
    ret = cam.IMV_FG_SetEnumFeatureValue("Binning", pEnumSymbol)
    if (IMV_FG_OK != ret):
        print("Set feature value failed! ErrorCode[%d]"% ret)
        return ret

// Configure the attribute value.
    ret = cam.IMV_FG_SetEnumFeatureValue("Binning", enumValue.value)
    if (IMV_FG_OK != ret):
        print("Set feature value failed! ErrorCode[%d]"% ret)
        return ret
```

401674 Iraypie 2025-01-18

3.5.4.5 String Attribute

Figure 3-5 String attribute



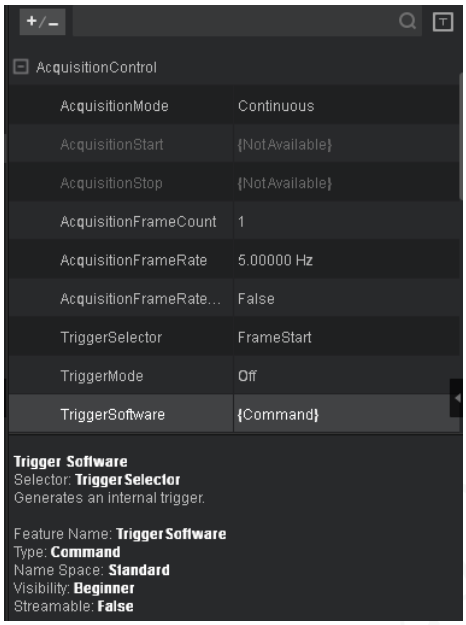

```
id = IMV_FG_String()

// Obtain attribute value.
ret = cam.IMV_FG_GetStringFeatureValue ("DeviceUserID", id)
if (IMV_FG_OK != ret):
    print("Get feature value failed! ErrorCode[%d]"% ret)
    return ret

// Configure the attribute value.
ret = cam.IMV_FG_SetStringFeatureValue("DeviceUserID", id)
if (IMV_FG_OK != ret):
    print("Set feature value failed! ErrorCode[%d]"% ret)
    return ret
```

3.5.4.6 Command Attribute

Figure 3-6 Command attribute




```
// Execute the attribute value of command.
ret = cam.IMV_FG_ExecuteCommandFeature ("TriggerSoftware")
if (IMV_FG_OK != ret):
    print("Execute Command feature failed! ErrorCode[%d]"% ret)
    return ret
```

3.6 Image Operation

3.6.1 Video Recording

- Interface: IMV_FG_OpenRecord(self,pstRecordParam)

Table 3-52 Parameter description

Parameter	I/O	Type	Description
self	IN	card	Acquisition card object.
pstRecordParam	IN	IMV_FG_RecordParam	Recording parameter struct.

  - ◇ Description: Start recording. If succeeded, return **IMV_FG_OK**. If failed, return error code.
  - ◇ Reference Routine: **Record**.
- Interface: IMV_FG_InputOneFrame(self,pstRecordFrameInfoParam)

Table 3-53 Parameter description

Parameter	I/O	Type	Description
self	IN	card	Acquisition card object.
pstRecordFrameInfoParam	IN	IMV_FG_RecordFrameInfoParam	Frame information structure for recording.

◇ Description: Record a frame of image. If succeeded, return **IMV_FG_OK**. If failed, return error code.

◇ Reference Routine: **Record**.

- Interface: IMV_FG_CloseRecord(self)

Table 3-54 Parameter description

Parameter	I/O	Type	Description
self	IN	card	Acquisition card object.

◇ Description: Stop Recording. If succeeded, return **IMV_FG_OK**. If failed, return error code.

◇ Reference Routine: **Record**.

3.6.2 Image Operation

- Interface: IMV_FG_PixelConvert(self,pstPixelConvertParam)

Table 3-55 Parameter description

Parameter	I/O	Type	Description
self	IN	card	Acquisition card object.
pstPixelConvertParam	IN/OUT	IMV_FG_PixelConvertParam	Pixel format conversion parameter structure.

◇ Description: Pixel format conversion. If succeeded, return **IMV_FG_OK**. If failed, return error code.

□ The supported target pixel format: **gvspPixelRGB8/ gvspPixelBGR8/ gvspPixelMono8/ gvspPixelBGRA8.**

- Convert the raw image data into the pixel format required by the user and stored in the specified memory by the caller.
- When pixel format is **YUV411Packed**, the image width must be divisible by 4.
- When pixel format is **YUV422Packed**, the image width must be divisible by 2.
- When pixel format is **YUYVPacked**, the image width must be divisible by 2.
- Converted image: Data storage starts from the first row at the top, which is the default storage direction of camera data.

◇ Reference Routine: **ImageConvert**, **SaveImageToBmp**.

- Interface: IMV_FG_FlipImage(self,pstFlipImageParam)

Table 3-56 Parameter description

Parameter	I/O	Type	Description
self	IN	card	Acquisition card object.
pstFlipImageParam	IN/OUT	IMV_FG_FlipImageParam	Image flip parameter structure.

◇ Description: Image flip. If succeeded, return **IMV_FG_OK**. If failed, return error code.

- The supported pixel format of the vertical and horizontal image flip: **gvspPixelRGB8**/ **gvspPixelBGR8**/ **gvspPixelMono8**.
- Convert the raw image data and store them in the specified memory by the caller.

◇ Reference Routine: **ImageFlip**.

- Interface: IMV_FG_RotateImage(self,pstRotateImageParam)

Table 3-57 Parameter description

Parameter	I/O	Type	Description
self	IN	card	Acquisition card object.
pstRotateImageParam	IN/OUT	IMV_FG_RotateImageParam	Image flip parameter structure.

◇ Description: clockwise rotation of the image. If succeeded, return **IMV_FG_OK**. If failed, return error code.

- The supported image format of the 90/180/270 degrees clockwise rotation: **gvspPixelRGB8** / **gvspPixelBGR8** / **gvspPixelMono8**.
- Rotate the raw image data and store them in the specified memory by the caller.

◇ Reference Routine: **ImageRotate**.

- Interface: IMV_FG_SaveImageToFile(self, pstSaveFileParam)

Table 3-58 Parameter description

Parameter	I/O	Type	Description
self	IN	card	Acquisition card object.
pstSaveFileParam	IN	IMV_FG_SaveImageToFileParam	Image saving parameter structure.

◇ Description: Save the image to the file. If succeeded, return **IMV_FG_OK**. If failed, return error code.

- The image format supported by the interface includes **BMP**/**JPEG**/**PNG**/**TIFF**.
- The supported max height-width size of JPEG format is 65,500.

◇ Reference Routine: **SaveImageToFile**.

401674 Iraypl 2025-01-16

4 Matched Routine Description

Python Routine Path: 【 SDK Installation Directory 】 \Development\ Samples\Python\IMVFG

Table 4-1 Routine description

Routine Name	Routine Description
Grab	Image Capturing in free-run mode (active streaming).
GrabCallback	Streaming callback.
ImageConvert	Image format conversion.
ImageFlip	Image flipping.
ImageRotate	Image rotation.
LoadAndSaveDeviceCfg	Save and load the attribute configuration file of device.
MultipleCamera	Multi-camera acquiring.
SaveImageToFile	Save image.
SaveImageToBmp	Save the BMP image.
SoftTrigger	Acquisition in software trigger mode (callback to acquire).

401674 Iraypl 2025-01-18

5 FAQ (Frequently Asked Question)

5.1 Only 8 Images are Obtained by Using the Python Interface

Reason

After acquiring the images using python interface, the images are not released resulting in the buffer memory being filled and preventing the acquisition of new images.

Solution

Every time the frame is acquired, the buffer memory shall be release by calling the interface of **IMV_FG_ReleaseFrame** as shown in the figure below.

Table 5-1 IMV_FG_ReleaseFrame interface

```
#get a frame image
nRet = card.IMV_FG_GetFrame(frame, 500)
if IMV_FG_OK != nRet:
    print("Get frame failed!ErrorCode", nRet)
    continue

print("Get frame blockId = [%d]"% frame.frameInfo.blockId)
```

```
#free image buffer
nRet = card.IMV_FG_ReleaseFrame(frame)
if IMV_FG_OK != nRet:
    print("Release frame failed! ErrorCode", nRet)
```